PARTE 4

Funções compostas e exercícios guiados

Encapsulando cálculos, condições e repetições

Aluno(a):	Antonio Maia de Jesus Chaves Neto	**Turma:**	Ciência da Computação_Noturno
Data:	27/09/2026	**Professor:**	Diego Abreu

Objetivos

- Criar funções booleanas para representar condições do problema.
- Construir funções que chamam outras funções.
- Integrar funções com decisões, repetição, validação e cálculos das aulas anteriores.

Atividade 1 - Função booleana: par ou ímpar

```
#include <stdbool.h>

bool ehPar(int numero) {
   return numero % 2 == 0;
}
```

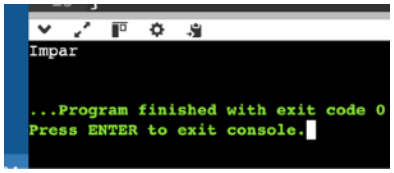
No main:

```
if (ehPar(numero)) {
   printf("Par\n");
} else {
   printf("Impar\n");
}
```

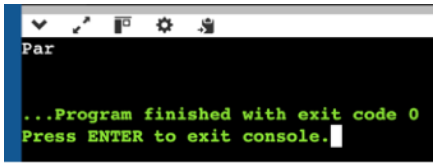
Teste com 2, 7, 0 e -4. Observe como o nome ehPar aproxima o código da linguagem do problema.



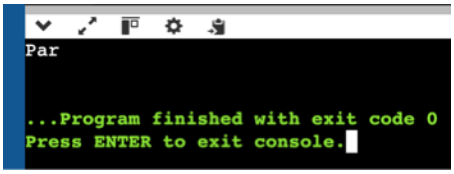
Teste: número = 7



Teste: número = 0



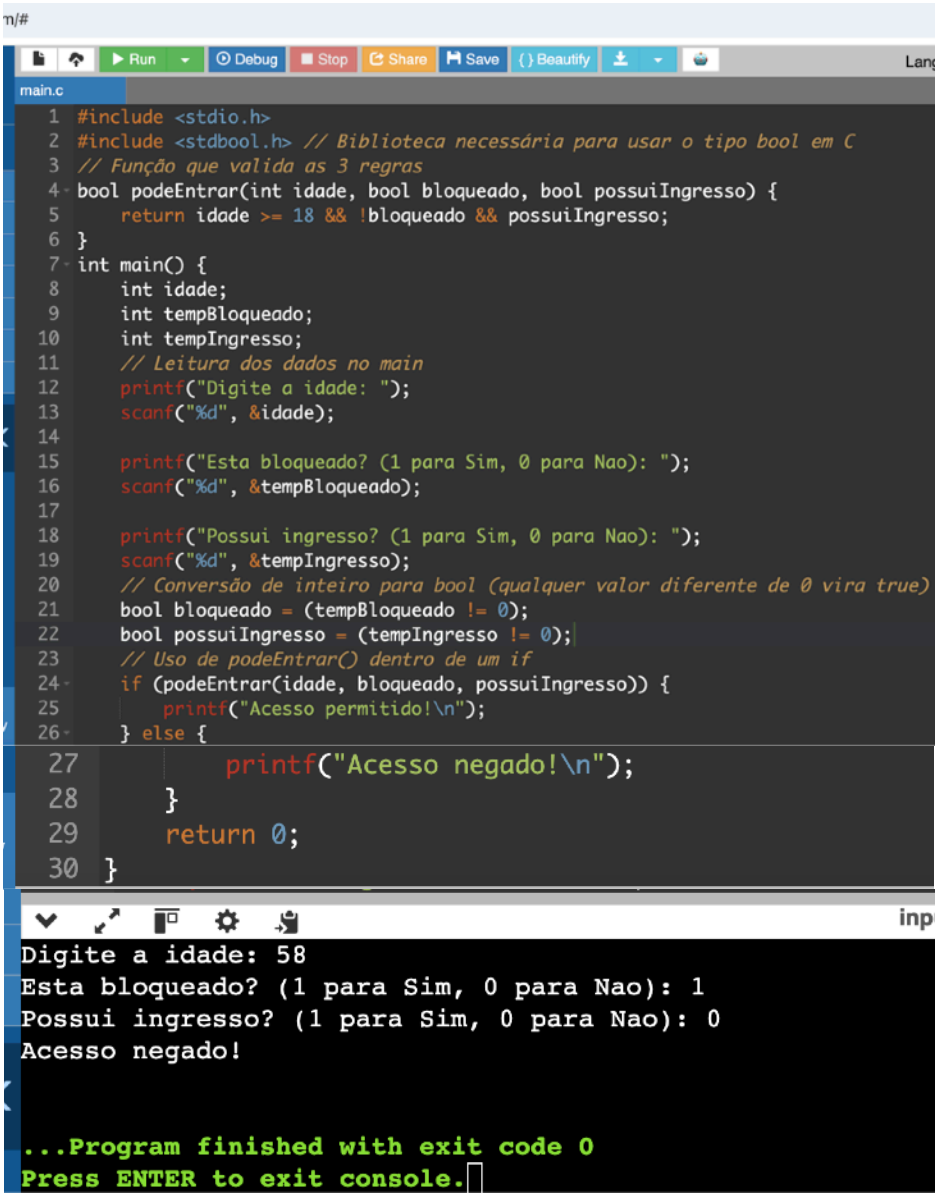
Teste: número = -4



Atividade 2 - Nomeando uma regra

```
bool podeEntrar(int idade, bool bloqueado) {
    return idade >= 18 && !bloqueado;
}
```

1. Leia idade e estado de bloqueio no main.
2. Use podeEntrar() dentro de um if.
3. Acrescente uma terceira regra: possuir ingresso.

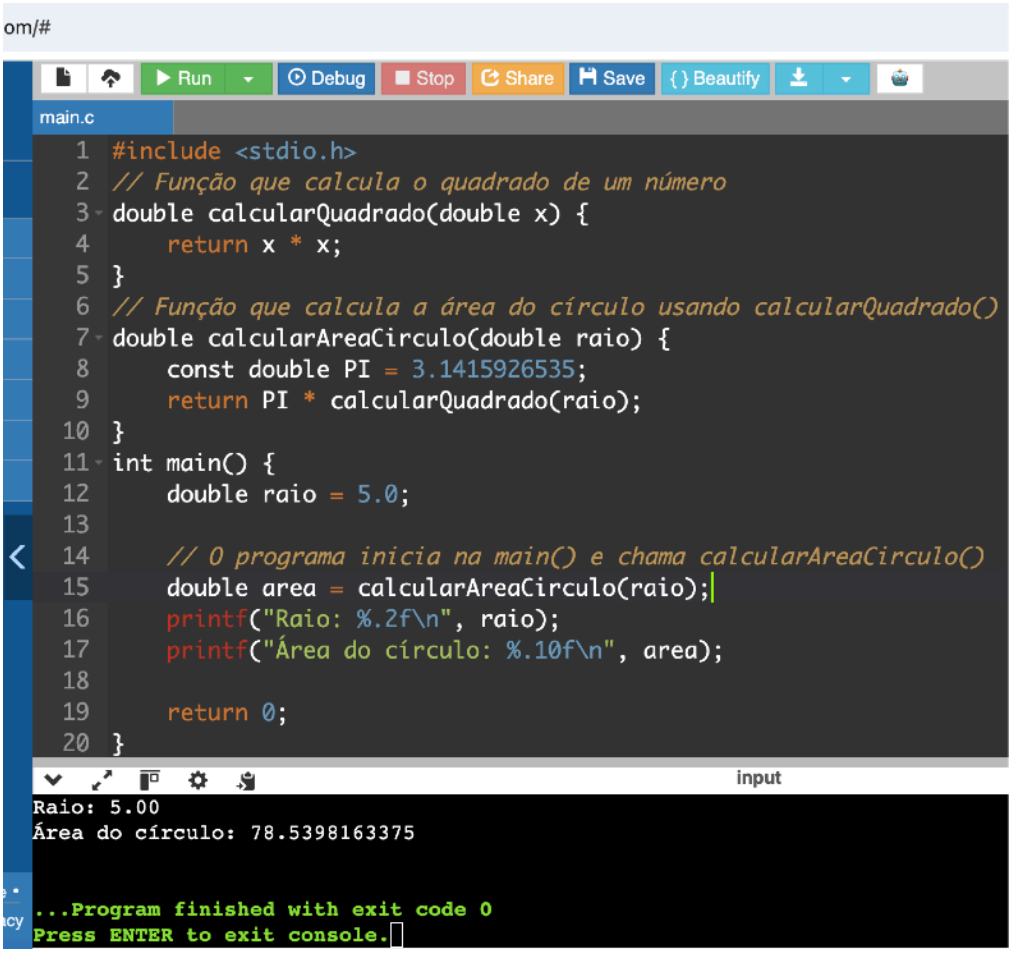

Run
Debug
Stop
Share
Save
{} Beautify
main.c
#include <stdio.h>
#include <stdbool.h> // Biblioteca necessária para usar o tipo bool em C
// Função que valida as 3 regras
bool podeEntrar(int idade, bool bloqueado, bool possuiIngresso) {
return idade >= 18 && !bloqueado && possuiIngresso;
}
int main() {
int idade;
int tempBloqueado;
int tempIngresso;
// Leitura dos dados no main
printf("Digite a idade: ");
scanf("%d", &idade);
printf("Esta bloqueado? (1 para Sim, 0 para Nao): ");
scanf("%d", &tempBloqueado);
printf("Possui ingresso? (1 para Sim, 0 para Nao): ");
scanf("%d", &tempIngresso);
// Conversão de inteiro para bool (qualquer valor diferente de 0 vira true)
bool bloqueado = (tempBloqueado != 0);
bool possuiIngresso = (tempIngresso != 0);
// Uso de podeEntrar() dentro de um if
if (podeEntrar(idade, bloqueado, possuiIngresso)) {
printf("Acesso permitido!\n");
} else {
printf("Acesso negado!\n");
}
return 0;
}
Digite a idade: 58
Esta bloqueado? (1 para Sim, 0 para Nao): 1
Possui ingresso? (1 para Sim, 0 para Nao): 0
Acesso negado!
...Program finished with exit code 0
Press ENTER to exit console.

Atividade 3 - Funções chamando funções

```
double calcularQuadrado(double x) {
    return x * x;
}

double calcularAreaCirculo(double raio) {
    const double PI = 3.1415926535;
    return PI * calcularQuadrado(raio);
}
```

Visualize o fluxo: main() chama calcularAreaCirculo(), que chama calcularQuadrado(). Quando a função interna termina, a execução retorna ao ponto anterior.



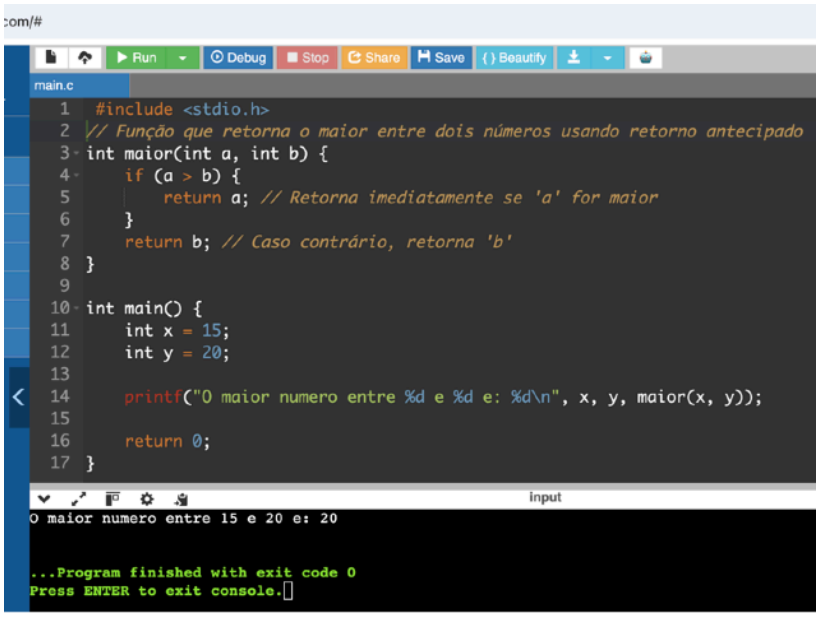
Pilha de chamadas: nesta aula basta entender conceitualmente que cada chamada suspende temporariamente a função atual e retorna a ela quando termina.

Atividade 4 - Retorno antecipado

```
bool notaValida(double nota) {
    if (nota < 0 || nota > 10) {
        return false;
    }
    return true;
}
```



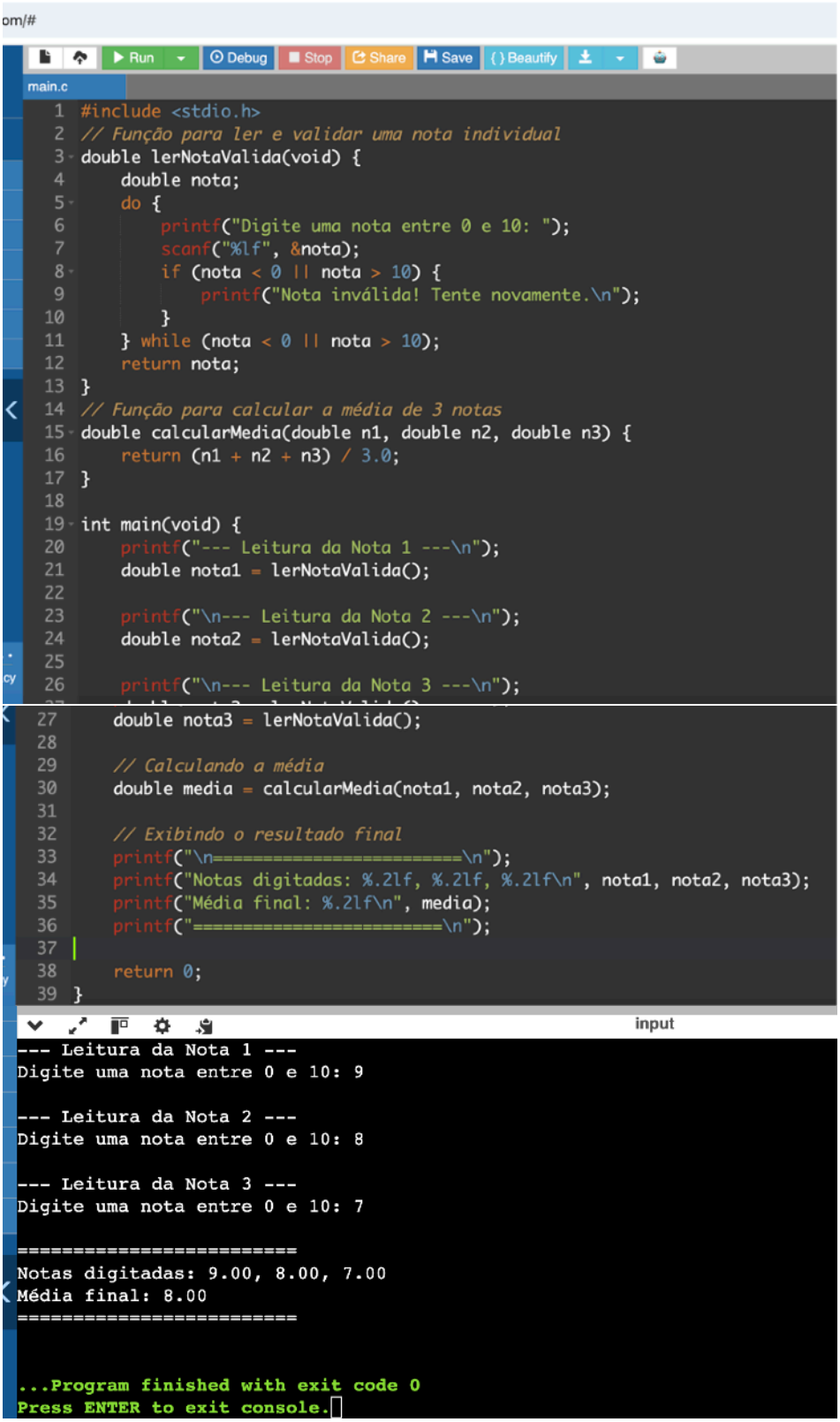
Crie também uma função int maior(int a, int b) usando retorno antecipado.



Atividade 5 - Leitura e validação dentro de função

```
double lerNotaValida(void) {
    double nota;
    do {
        printf("Digite uma nota entre 0 e 10: " );
        scanf("%lf", &nota);
    } while (nota < 0 || nota > 10);
    return nota;
}
```

Use a função três vezes e calcule a média com calcularMedia(). Aqui aparecem função, do-while, scanf, condição e retorno no mesmo programa.


Run
Debug
Stop
Share
Save
Beautify
main.c
#include <stdio.h>
// Função para ler e validar uma nota individual
double lerNotaValida(void) {
double nota;
do {
printf("Digite uma nota entre 0 e 10: ");
scanf("%lf", &nota);
if (nota < 0 || nota > 10) {
printf("Nota inválida! Tente novamente.\n");
}
} while (nota < 0 || nota > 10);
return nota;
}
// Função para calcular a média de 3 notas
double calcularMedia(double n1, double n2, double n3) {
return (n1 + n2 + n3) / 3.0;
}
int main(void) {
printf("--- Leitura da Nota 1 ---\n");
double nota1 = lerNotaValida();
printf("\n--- Leitura da Nota 2 ---\n");
double nota2 = lerNotaValida();
printf("\n--- Leitura da Nota 3 ---\n");
double nota3 = lerNotaValida();
// Calculando a média
double media = calcularMedia(nota1, nota2, nota3);
// Exibindo o resultado final
printf("\n========================\n");
printf("Notas digitadas: %.2lf, %.2lf, %.2lf\n", nota1, nota2, nota3);
printf("Média final: %.2lf\n", media);
printf("========================\n");
return 0;
}
input
--- Leitura da Nota 1 ---
Digite uma nota entre 0 e 10: 9
--- Leitura da Nota 2 ---
Digite uma nota entre 0 e 10: 8
--- Leitura da Nota 3 ---
Digite uma nota entre 0 e 10: 7
========================
Notas digitadas: 9.00, 8.00, 7.00
Média final: 8.00
========================
...Program finished with exit code 0
Press ENTER to exit console.

Exercícios guiados

Exercício A - Maior e menor

Implemente int maior(int a, int b) e int menor(int a, int b). Leia dois números e teste ambas.

Exercício B - Conversões

Implemente celsiusParaFahrenheit(), fahrenheitParaCelsius() e kmParaMilhas().

Exercício C - Sistema acadêmico

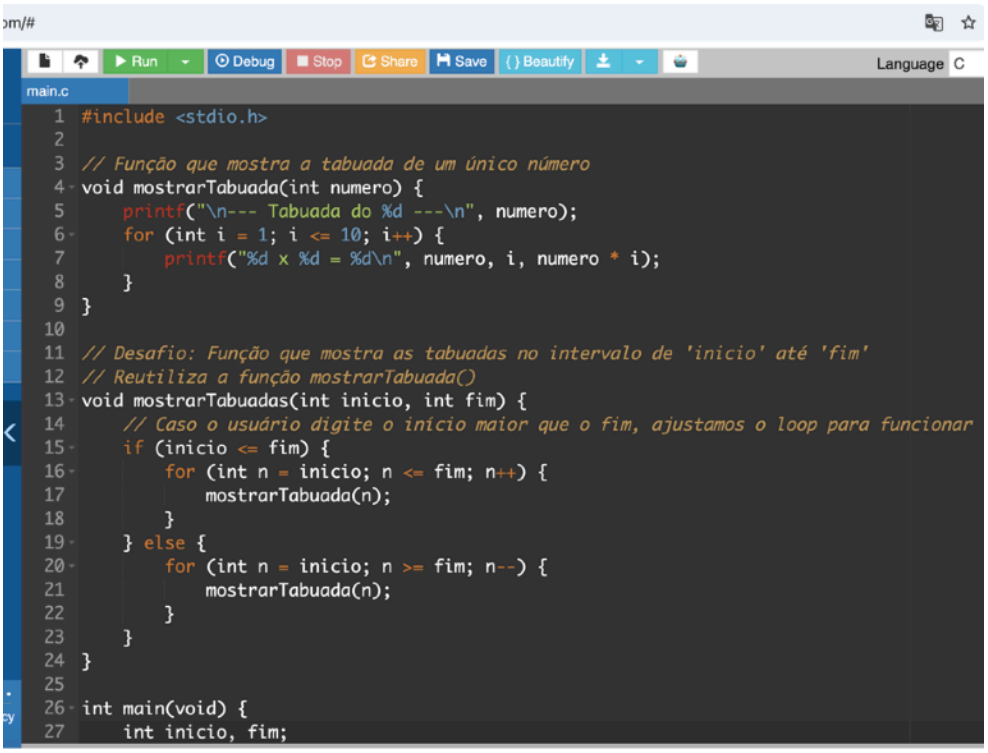
```
double calcularMedia(double n1, double n2, double n3);
bool aprovado(double media, double frequencia);
void mostrarResultado(double media, double frequencia);
```

Regra: média >= 7 e frequência >= 75. Separe claramente cálculo, regra e apresentação.

Exercício D - Tabuada como função

```
void mostrarTabuada(int numero) {
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", numero, i, numero * i);
    }
}
```

Desafio: implemente void mostrarTabuadas(int inicio, int fim) e reutilize mostrarTabuada().



```c
    printf("   GERADOR DE TABUADAS EM C   \n");

    printf("Digite o número inicial da tabuada: ");
    scanf("%d", &inicio);

    printf("Digite o número final da tabuada: ");
    scanf("%d", &fim);

    // Chamada da função que reutiliza mostrarTabuada()
    mostrarTabuadas(inicio, fim);

    return 0;
}
```

```
--- Tabuada do 7 ---
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

--- Tabuada do 8 ---
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80

--- Tabuada do 9 ---
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

4. Qual vantagem existe em encapsular numero % 2 == 0 dentro de ehPar()?

Resposta:

A principal vantagem é a legibilidade, a manutenibilidade e o princípio *Don't Repeat Yourself(DRY)*. Se a lógica de verificação de paridade mudar ou precisar ser aplicada em vários lugares. Assim, recisa-se alterar o código em um único lugar (ehPar), além de dar um nome semântico claro à operação.

5. O que acontece com a execução quando uma função chama outra?

Resposta:

A função atual (chamadora) pausa temporariamente a sua execução. O programa armazena o ponto de retorno e os dados locais na pilha de chamadas (call stack), e o fluxo de controle é transferido para a nova função (chamada). Quando a função chamada termina, o controle retorna para a função original exatamente no ponto de onde parou.

6. Por que lerNotaValida() é mais reutilizável do que repetir o do-while em vários pontos?

Resposta:

Porque centraliza a lógica de validação (como tratar valores inválidos, mensagens de erro e limites de notas) em um único bloco de código. Se as regras da nota mudarem futuramente, você faz a correção em apenas uma função, evitando a duplicação de código e reduzindo a chance de erros ou inconsistências no programa.

Entrega da parte

- Funções ehPar(), podeEntrar() e calcularAreaCirculo().
- Sistema acadêmico com três responsabilidades separadas.
- mostrarTabuadas() reutilizando mostrarTabuada().